



Abschlussprüfung Sommer 2026

Fachinformatiker für Anwendungsentwicklung
Dokumentation zur betrieblichen Projektarbeit

Entwicklung des AntispamAdminCenter

Mandantenfähige Webanwendung zur Verwaltung von Mail- und IP-Freigaben

Abgabetermin: Strausberg, den 23.04.2026

Prüfungsbewerber:

Klaus Bindemann
Kastanienweg 2b
15377 Oberbarnim



villadata
systemhaus GmbH

Ausbildungsbetrieb:

AVADO - VILLADATA Systemhaus GmbH
Am Flugplatz 12a
15344 Strausberg

Inhaltsverzeichnis

Abkürzungsverzeichnis	III
1 Das Thema und Problemstellung	1
1.1 Zielstellung der Projektarbeit	1
1.2 Ist-Analyse	1
1.3 Soll-Konzept	2
1.4 Projektabgrenzung	2
1.5 Projektorganisation und Zeitplanung	2
2 Technisch-wirtschaftlicher Variantenvergleich	3
2.1 Technische Bewertung der Varianten	3
2.2 Wirtschaftliche Bewertung	4
2.3 Nutzwertanalyse und Entscheidung	5
3 Durchführung und Ergebnisse	5
3.1 Architektur und Zielplattform	5
3.2 Entwurf von Datenmodell und Rechtestruktur	5
3.3 Entwurf der Benutzeroberfläche	6
3.4 Sicherheitskonzept und Auditierung	6
3.5 Schnittstellen und Datenfluss	6
3.6 Implementierung fachlicher Kernfunktionen	7
3.7 Implementierung der Benutzer- und Mandantenverwaltung	7
3.8 Erzielte Projektergebnisse	7
4 Qualitätssicherung	8
4.1 Soll-Ist-Vergleich von Leistung, Zeit und Inhalt	8
4.2 Bewertung der Testergebnisse	9
5 Kundendokumentation	9
6 Fazit	10
Literatur- und Quellenverzeichnis	11
Persönliche Erklärung	12
A Anhang	i
A.1 Zeitnachweis	i
A.2 Wirtschaftlichkeit	ii
A.3 Variantenvergleich	iii
A.4 Use Case-Diagramm	iv
A.5 Datenbankmodell	v
A.6 Klassendiagramm	vi

A.7 Komponentendiagramm	vii
A.8 Sequenzdiagramm Login	viii
A.9 Kundenhinweis	ix
A.10 Quelltextauszug Berechtigungslogik	x
A.11 Testauszug	xiii

Abkürzungsverzeichnis

API Application Programming Interface

CRUD Create, Read, Update, Delete

HTML Hypertext Markup Language

HTTP Hypertext Transfer Protocol

IP Internet Protocol

ORM Object-Relational Mapping

SQL Structured Query Language

UI User Interface

UML Unified Modeling Language

1 Das Thema und Problemstellung

Der Ausbildungsbetrieb betreibt und betreut Mail-Infrastrukturen für mehrere Organisationen. In diesem Umfeld müssen freigegebene Mail-Adressen und IP-Adressen regelmäßig angepasst werden, damit technische Ausnahmen im Antispam-Umfeld nachvollziehbar und wirksam gepflegt bleiben. Vor Beginn des Projekts erfolgte diese Pflege ohne zentrale Anwendung. Informationen waren auf mehrere Datenquellen verteilt, Änderungen wurden manuell übernommen und Rückfragen mussten häufig im direkten Austausch geklärt werden.

Die Ausgangssituation war fachlich problematisch, weil dieselben Daten an verschiedenen Stellen gepflegt wurden und keine einheitliche Bedienlogik vorhanden war. Dadurch entstanden Medienbrüche, uneinheitliche Datenstände und ein zusätzlicher Abstimmungsaufwand zwischen den beteiligten Mitarbeitern. Gleichzeitig war nur eingeschränkt nachvollziehbar, wer eine Freigabe angelegt, geändert oder entfernt hatte. Für einen sicherheitsnahen Fachprozess ist dieser Zustand langfristig nicht tragfähig.

1.1 Zielstellung der Projektarbeit

Ziel der Projektarbeit war die Entwicklung des *AntispamAdminCenter*, einer webbasierten Verwaltungsanwendung für die zentrale Pflege von Organisationen, Benutzern, Mail-Adressen und IP-Adressen. Die Anwendung sollte von internen Administratoren und autorisierten Organisationsbenutzern genutzt werden können und dabei eine eindeutige Trennung zwischen globalen und mandantenbezogenen Rechten sicherstellen.

Aus dem Projektauftrag ergaben sich folgende Kernziele:

- zentrale und einheitliche Pflege aller fachlich relevanten Stammdaten
- rollenbasiertes Berechtigungskonzept mit Mandantentrennung
- abgesicherte Anmeldung mit Session-Verwaltung und Passwort-Hashing
- nachvollziehbare Protokollierung fachlicher Änderungen
- Bereitstellung einer belastbaren Datenbasis für nachgelagerte Mail-Systeme

Das Projektergebnis sollte nicht nur die Bearbeitungszeit senken, sondern vor allem die Qualität der Datenhaltung verbessern. Neben der fachlichen Funktion stand daher die Beherrschbarkeit des Betriebsprozesses im Vordergrund.

1.2 Ist-Analyse

Die Ist-Analyse zeigte, dass die bisherige Vorgehensweise stark personenabhängig war. Änderungen wurden zwar fachlich abgestimmt, technisch jedoch nicht über eine dafür vorgesehene Anwendung umgesetzt. Daraus ergaben sich drei zentrale Schwachstellen.

Erstens fehlte eine zentrale Datenhaltung für freigegebene Mail- und IP-Einträge. Zweitens war die Berechtigungsstruktur nicht systematisch abgebildet, so dass zwischen

globalen Administratorrechten und organisationsbezogenen Zuständigkeiten nur eingeschränkt unterschieden werden konnte. Drittens existierte keine belastbare Historie, über die sich Änderungen und deren Verursacher im Nachhinein nachvollziehen ließen.

Aus betrieblicher Sicht führte dies zu wiederkehrenden Rückfragen, manuellen Kontrollen und vermeidbaren Fehlerquellen. Die eigentliche Pflegeaufgabe war zwar fachlich überschaubar, der organisatorische Aufwand rund um Kontrolle, Abstimmung und Nacharbeit fiel jedoch unverhältnismäßig hoch aus.

1.3 Soll-Konzept

Im Soll-Zustand soll eine zentrale Webanwendung den gesamten Pflegeprozess unterstützen. Organisationen, Benutzer, Mail-Adressen und IP-Adressen werden in einer gemeinsamen Datenbasis verwaltet und über einheitliche Formulare gepflegt. Rollen und Organisationszuordnung steuern, welche Daten angezeigt und geändert werden dürfen. Fachliche Änderungen werden revisionsfähig im Audit-Log festgehalten.

Technisch wurde eine schlanke Webanwendung auf Basis von FastAPI, SQLAlchemy und SQLite vorgesehen. Diese Kombination war für den Projektumfang geeignet, weil sie eine schnelle Umsetzung, klare Trennung zwischen Datenmodell, Fachlogik und Webschicht sowie einfache Testbarkeit ermöglicht. Nicht Bestandteil des Projekts sind die eigentliche Spam-Filterlogik, ein produktionsreifes Mehrserver-Deployment oder die Migration beliebiger Altdatenbestände.

1.4 Projektabgrenzung

Das Projekt umfasst die Entwicklung einer intern nutzbaren Verwaltungsanwendung einschließlich Rollenmodell, Auditierung und Kundendokumentation. Nicht Gegenstand der Projektarbeit sind der produktive Betrieb einer hochverfügbaren Infrastruktur, eine allgemeine Mandantenmigration aus Fremdsystemen oder die technische Umsetzung aller denkbaren Schnittstellen zu nachgelagerten Mail-Systemen. Diese Abgrenzung war erforderlich, um den Projektumfang auf einen in 80 Stunden realisierbaren und bewertbaren Kern zu konzentrieren.

1.5 Projektorganisation und Zeitplanung

Die Projektdurchführung erfolgte im für die FIAE-Projektarbeit vorgesehenen Zeitbudget von 80 Stunden. Zur Steuerung des Vorhabens wurde die Arbeit in Analyse, Soll-Konzept, Entwurf, Implementierung, Test, Inbetriebnahmevorbereitung und Dokumentation gegliedert. Diese Aufteilung war notwendig, damit sowohl die fachliche Problemlösung als auch die Qualitätssicherung innerhalb des vorgegebenen Rahmens belastbar geplant werden konnten.

Projektphase	Stunden
Analyse der Ausgangssituation und Anforderungsklä- rung	8
Soll-Konzept und Variantenvergleich	10
Entwurf von Datenmodell, Rollenmodell und Oberflä- che	12
Implementierung der Anwendung	28
Tests, Fehlerkorrekturen und Abnahmevorbereitung	10
Kundendokumentation und Projektdokumentation	12
Summe	80

Tabelle 1: Geplante Zeitaufteilung für das Projekt

Die Umsetzung erfolgte iterativ. Nach der Erhebung der fachlichen Anforderungen wurden die Teilbereiche Organisationen, Benutzer, Mail-Adressen, IP-Adressen, Authentifizierung und Auditierung zunächst konzeptionell getrennt und danach schrittweise implementiert. Dieses Vorgehen war zweckmäßig, weil die fachliche Rückkopplung aus dem internen IT-Umfeld unmittelbar in Entwurf und Implementierung übernommen werden konnte.

Materiell wurde ein vorhandener Entwicklungsarbeitsplatz mit Python 3.12, FastAPI, SQL-Model, SQLite und einer lokalen Testumgebung genutzt. Externe Lizenzkosten fielen für die im Projekt eingesetzten Kernkomponenten nicht an.

2 Technisch-wirtschaftlicher Variantenvergleich

Für die Zielerreichung wurden drei Lösungsvarianten betrachtet. Variante 1 war die Beibehaltung der manuellen Pflege. Variante 2 war die Einführung einer allgemeinen Standardlösung für Stammdaten- und Freigabeverwaltung. Variante 3 war die Entwicklung einer betriebspezifischen Webanwendung. Die Entscheidung wurde sowohl technisch als auch wirtschaftlich bewertet.

2.1 Technische Bewertung der Varianten

Die manuelle Pflege verursacht keine Einführungskosten, löst jedoch keine der identifizierten Kernprobleme. Weder die Mandantentrennung noch eine nachvollziehbare Änderungshistorie werden dadurch systematisch abgebildet. Auch Validierungen und ein konsistentes Rollenmodell fehlen weiterhin.

Eine Standardlösung reduziert den manuellen Aufwand grundsätzlich, wäre für den konkreten Einsatzzweck jedoch nur eingeschränkt passend. Der benötigte Funktionsumfang ist zwar überschaubar, kombiniert aber mehrere spezifische Anforderungen: organisationsbezogene Rechte, Verwaltung von Mail- und IP-Freigaben, Session-Login, Audit-Log und Datenbereitstellung für die nachgelagerte Mail-Infrastruktur. Bei einer Fremdlösung wären deshalb Anpassungen, Schnittstellenabstimmungen und ein fachlicher Kompromiss beim Rollenmodell zu erwarten gewesen.

Die Eigenentwicklung deckt den benötigten Funktionsumfang zielgerichtet ab und lässt sich an den vorhandenen Betriebsprozess anpassen. Gleichzeitig bleibt die Anwendung technisch überschaubar, weil sie nur die fachlich benötigten Funktionen enthält. Der zusätzliche Entwicklungsaufwand ist für den Projektumfang vertretbar und führt zu einer wartbaren Lösung ohne unnötige Abhängigkeit von Fremdprodukten.

Kriterium	Manuell	Standardlösung	Eigenentwicklung
Mandantentrennung	schwach	mittel	hoch
Fachprozessabbildung	schwach	mittel	hoch
Nachvollziehbarkeit von Änderungen	schwach	mittel	hoch
Anpassbarkeit	niedrig	mittel	hoch
Betriebliche Einführbarkeit	hoch	mittel	hoch

Tabelle 2: Technischer Vergleich der Lösungsvarianten

2.2 Wirtschaftliche Bewertung

Die wirtschaftliche Betrachtung basiert auf typischen Pflegevorgängen im Administrationsalltag. Für die Kalkulation wurden 60 Änderungsvorgänge pro Monat angesetzt. Vor der Projektdurchführung betrug der mittlere Aufwand pro Vorgang rund 10 Minuten, da Informationen zusammengesucht, Zuständigkeiten abgestimmt und Eingaben teilweise manuell nachgehalten werden mussten. Mit der neuen Anwendung sinkt der Bearbeitungsaufwand auf rund 4 Minuten, weil Suchaufwand, Mehrfacheingaben und Rückfragen reduziert werden.

Kennzahl	Wert
Pflegevorgänge pro Monat	60
Bearbeitungszeit vorher je Vorgang	10 min
Bearbeitungszeit nachher je Vorgang	4 min
Zeitersparnis je Vorgang	6 min
Zeitersparnis pro Monat	360 min = 6 h

Tabelle 3: Grundlage der Wirtschaftlichkeitsbetrachtung

Für die Projektkosten wurde kein Stundenlohn, sondern ein interner kalkulatorischer Verrechnungssatz angesetzt. Für die 80 Projektstunden wurden 65 € pro Stunde angesetzt. Zusätzlich wurden sechs Stunden fachliche Begleitung und Abnahme mit 85 € pro Stunde bewertet. Daraus ergeben sich Projektkosten von 5710 €. Eine Detailrechnung ist im Anhang A.2: Wirtschaftlichkeit dargestellt.

Bei einer monatlichen Einsparung von 6 Stunden und einem kalkulatorischen internen Satz von 65 € ergibt sich ein wirtschaftlicher Nutzen von 390 € pro Monat. Die Projektkosten amortisieren sich damit nach rund 14,6 Monaten. Die Amortisationsdauer liegt für eine intern genutzte Verwaltungsanwendung im vertretbaren Bereich, weil neben der Zeitersparnis auch qualitative Effekte wie Nachvollziehbarkeit, Fehlervermeidung und Mandantentrennung erreicht werden.

2.3 Nutzwertanalyse und Entscheidung

Da sich die Entscheidung nicht allein über die Bearbeitungszeit ableiten lässt, wurde zusätzlich eine Nutzwertanalyse durchgeführt. Bewertet wurden die Kriterien Nachvollziehbarkeit, Sicherheit, Passgenauigkeit des Funktionsumfangs, Bedienbarkeit und Wartbarkeit. Die gewichtete Auswertung ist im Anhang A.3: Variantenvergleich enthalten.

Die manuelle Pflege erzielt den niedrigsten Nutzwert, weil sie den bestehenden Problembestand unverändert fortschreibt. Die Standardlösung verbessert die Bedienung, erreicht jedoch nicht ohne Zusatzaufwand die geforderte Passgenauigkeit. Die Eigenentwicklung erzielt den höchsten Nutzwert, weil sie den Fachprozess zielgerichtet abbildet, technisch beherrschbar bleibt und ohne funktionsfremde Bestandteile eingeführt werden kann. Aus diesen Gründen wurde die Eigenentwicklung als wirtschaftlich und technisch sinnvollste Variante ausgewählt.

Für die Entscheidung war ausschlaggebend, dass die wirtschaftliche Bewertung nicht isoliert betrachtet wurde. Gerade im vorliegenden Umfeld wirken sich fehlerhafte Freigaben oder unklare Zuständigkeiten nicht nur auf die Bearbeitungszeit, sondern auch auf die Betriebssicherheit aus. Die Eigenentwicklung bietet deshalb den größten Gesamtnutzen, obwohl sie in der Einführungsphase höhere Initialkosten verursacht als die Fortführung des Bestandszustands.

3 Durchführung und Ergebnisse

3.1 Architektur und Zielplattform

Die Anwendung wurde als serverseitige Webanwendung auf Basis von Python 3.12, FastAPI und SQLAlchemy umgesetzt. Der Zugriff erfolgt über einen Browser. Eine separate Client-Installation ist daher nicht erforderlich. Dieses Modell war für den internen Einsatz zweckmäßig, weil die Anwendung ohne aufwendige Verteilung auf Arbeitsplatzrechner bereitgestellt werden kann.

Die technische Struktur ist in vier Bereiche gegliedert: Datenmodell, Fachlogik, Web-API und Templates. Im Quellcode entsprechen diese Bereiche den Ordnern `app/models`, `app/services`, `app/web/api` und `app/web/templates`. Dadurch bleiben Datenmodell, Request-Verarbeitung und Darstellung sauber getrennt. Diese Trennung vereinfacht die Wartung und erleichtert automatisierte Tests, weil Fachregeln nicht in einzelnen Templates oder Routenfunktionen verteilt sind. Ein Komponentendiagramm ist im Anhang A.7: Komponentendiagramm dargestellt.

3.2 Entwurf von Datenmodell und Rechtestruktur

Das Datenmodell umfasst die fachlichen Entitäten Organisation, Benutzer, Mail-Adresse und IP-Adresse. Ergänzt werden diese durch Audit- und Login-Tabellen. Damit lassen

sich sowohl fachliche Änderungen als auch sicherheitsrelevante Login-Vorgänge nachvollziehen.

Im Berechtigungskonzept wurden globale und organisationsbezogene Rechte getrennt. Root- und Admin-Rollen dürfen administrative Aufgaben für mehrere Organisationen ausführen, während einfache Benutzer auf die ihnen zugeordneten Mandanten begrenzt bleiben. Diese fachliche Trennung war notwendig, damit die Anwendung in einem mehrmandantenfähigen Umfeld sicher eingesetzt werden kann. Das Datenbankmodell ist im Anhang A.5: Datenbankmodell dokumentiert.

3.3 Entwurf der Benutzeroberfläche

Die Benutzeroberfläche wurde bewusst schlank gehalten. Im Mittelpunkt stand eine fehlerarme Datenerfassung und nicht die Darstellung technischer Details. Die Navigation führt direkt zu den Kernbereichen Dashboard, Organisationen, Benutzer, Mail-Adressen, IP-Adressen und Audit-Log. Die einzelnen Masken sind formularbasiert aufgebaut und orientieren sich an den täglichen Pflegevorgängen des Fachprozesses.

Validierungsfehler werden unmittelbar an den betroffenen Eingabefeldern angezeigt. Dadurch werden falsche E-Mail-Adressen, ungültige IP-Adressen oder unvollständige Stammdaten nicht erst im Nachgang erkannt. Die klare und in allen Bereichen einheitliche Bedienlogik reduziert den Einarbeitungsaufwand und senkt die Wahrscheinlichkeit fehlerhafter Eingaben.

3.4 Sicherheitskonzept und Auditierung

Neben der fachlichen Bedienung wurde die Anwendung mit einem einfachen, aber konsequenten Sicherheitskonzept entworfen. Dazu gehören Session-Login, Passwort-Hashing, Login-Audit, Rate-Limiting bei Fehlanmeldungen sowie kontobezogene Sperrmechanismen. Diese Funktionen sollen verhindern, dass berechtigungspflichtige Verwaltungsfunktionen unkontrolliert genutzt werden.

Die Auditierung wurde bewusst als eigener Fachbaustein behandelt. Jede relevante Änderung an Organisationen, Benutzern, Mail-Adressen und IP-Adressen wird mit Kontextinformationen protokolliert. Dadurch können fehlerhafte Pflegevorgänge nachvollzogen und fachlich bewertet werden. Für ausgewählte Szenarien ist außerdem ein Rollback vorgesehen, um unbeabsichtigte Änderungen kontrolliert rückgängig machen zu können.

3.5 Schnittstellen und Datenfluss

Die Anwendung stellt sowohl HTML-basierte Oberflächen als auch schlanke API-Endpunkte bereit. Dadurch können Daten direkt über den Browser gepflegt werden, während die fachlich verwalteten Mail- und IP-Informationen zugleich in strukturierter Form für nachgelagerte Verarbeitungsschritte bereitstehen. Dieses Modell vermeidet

doppelte Datenhaltung und reduziert Medienbrüche zwischen Pflege und technischer Weiterverarbeitung.

Im Datenfluss beginnt jeder Pflegevorgang mit der Authentifizierung des Benutzers. Danach prüft die Anwendung Rolle und Organisationszuordnung, validiert die Eingaben, schreibt die Änderung in die Datenbank und erzeugt bei fachlich relevanten Aktionen einen Audit-Eintrag. Dieser Ablauf stellt sicher, dass Anzeige, Berechtigung, Speicherung und Nachvollziehbarkeit konsistent zusammenwirken.

3.6 Implementierung fachlicher Kernfunktionen

Die Umsetzung erfolgte modulweise entlang der fachlichen Teilbereiche. Zunächst wurden die SQLModel-Klassen für Organisationen, Benutzer, Mail-Adressen, IP-Adressen und Audit-Daten angelegt. Anschließend wurden die zugehörigen Service-Module für Validierung, Benutzerverwaltung, Rollenprüfung und fachliche Pflegeprozesse implementiert. Dieses Vorgehen stellte sicher, dass die Webschicht auf klar abgegrenzte Fachfunktionen zugreifen kann.

Ein technischer Schwerpunkt lag auf der Authentifizierung. Die Anwendung nutzt Session-Login, Passwort-Hashing mit Argon2, Login-Audit sowie Schutzmechanismen gegen wiederholte Fehlanmeldungen. Damit wurde der Anforderung Rechnung getragen, dass sicherheitsnahe Verwaltungsfunktionen nicht über ungesicherte Standardzugriffe bereitgestellt werden dürfen.

3.7 Implementierung der Benutzer- und Mandantenverwaltung

Die Benutzer- und Mandantenverwaltung bildet den Kern des Fachprozesses. Organisationen können angelegt, geändert und logisch getrennt verwaltet werden. Benutzer erhalten Rollen und werden bestimmten Organisationen zugeordnet. Auf dieser Grundlage steuert die Anwendung, welche Datensätze eingesehen oder gepflegt werden dürfen.

Für Mail- und IP-Einträge wurden eigene Verwaltungsfunktionen umgesetzt, die Eingaben validieren und Änderungen protokollieren. Das vermeidet fachlich ungültige Daten und schafft eine einheitliche Grundlage für die nachgelagerte Weiterverarbeitung im Mail-Umfeld.

3.8 Erzielte Projektergebnisse

Mit Abschluss der Implementierung lag eine lauffähige Webanwendung vor, die den vorgesehenen Funktionsumfang abdeckt. Organisationen, Benutzer, Mail-Adressen und IP-Adressen können zentral verwaltet werden. Fachliche Änderungen werden im Audit-Log dokumentiert und können für definierte Fälle über einen Rollback wiederhergestellt werden.

Zusammenfassend wurde damit nicht nur eine neue Eingabemaske geschaffen, sondern ein strukturierter Verwaltungsprozess mit Rechtekonzept, Validierung und Historisierung eingeführt. Ein Quelltextauszug zur Berechtigungslogik befindet sich im Anhang A.10: Quelltextauszug Berechtigungslogik.

Während der Implementierung zeigte sich, dass insbesondere die Trennung zwischen organisationsbezogenen und globalen Rechten sauber im Service-Layer abgebildet werden muss. Eine reine Prüfung auf Oberflächenebene wäre nicht ausreichend gewesen, da schutzbedürftige Aktionen auch auf Routenniveau abgesichert werden müssen. Die Umsetzung im Service-Layer war daher ein wesentlicher Beitrag zur technischen Qualität des Ergebnisses.

4 Qualitätssicherung

Die Qualitätssicherung erfolgte durch die Kombination aus automatisierten und manuellen Tests. Auf Code-Ebene wurden insbesondere Validierungen, Benutzerverwaltung, Audit-Funktionen und Formularlogik über vorhandene Pytest-Tests abgesichert. Auf Anwendungsebene wurden die zentralen Arbeitsabläufe über die Weboberfläche geprüft, darunter Anmeldung, Rollenvergabe, Mandantentrennung sowie die Pflege von Mail- und IP-Adressen.

Die Teststrategie war für das Projekt ausreichend, weil sie sowohl die fachliche Logik als auch die Benutzerinteraktion abdeckt. Automatisierte Tests reduzieren das Risiko von Regressionen bei künftigen Anpassungen. Die manuellen Durchläufe waren zusätzlich erforderlich, um Sichtbarkeiten, Navigation und Fehlermeldungen im Zusammenspiel der einzelnen Komponenten zu prüfen. Ein Testauszug befindet sich im Anhang A.11: Testauszug.

4.1 Soll-Ist-Vergleich von Leistung, Zeit und Inhalt

Die geplanten Kernleistungen wurden erreicht. Die Anwendung ist mehrmandantenfähig, stellt ein rollenbasiertes Berechtigungskonzept bereit und ermöglicht die zentrale Pflege von Organisationen, Benutzern, Mail-Adressen und IP-Adressen. Zusätzlich wurden Audit-Log, Login-Schutz und Rollback für ausgewählte Änderungen umgesetzt.

Aspekt	Soll	Ist
Mandantenfähige Verwaltung	umgesetzt	umgesetzt
Rollen- und Rechtesystem	umgesetzt	umgesetzt
Auditierbare Änderungen	umgesetzt	umgesetzt
Rollback einzelner Änderungen	optional	umgesetzt
Projektzeit	80 h	eingehalten

Tabelle 4: Soll-Ist-Vergleich

Inhaltlich ergaben sich keine grundlegenden Abweichungen vom Projektziel. Der Schwerpunkt verlagerte sich im Projektverlauf leicht zugunsten von Sicherheits- und Nachvollziehbarkeitsanforderungen. Diese Vertiefung war fachlich sinnvoll, da gerade bei einer zentralen Verwaltungsanwendung die korrekte Rechteprüfung und die Historisierung der Änderungen für den späteren Betrieb entscheidend sind.

4.2 Bewertung der Testergebnisse

Die durchgeführten Tests bestätigten, dass die wesentlichen Pflegeabläufe stabil funktionieren. Insbesondere die Kombination aus Login, Rollenprüfung, Mandantentrennung und Auditierung konnte ohne fachliche Fehlfunktion nachgewiesen werden. Restpunkte betreffen keine fehlenden Kernfunktionen, sondern mögliche spätere Erweiterungen wie weitergehende Integrations- oder Lasttests für einen produktionsnahen Ausbau.

5 Kundendokumentation

Die Anwendung wird über einen Webbrowser aufgerufen. Nach erfolgreicher Anmeldung stehen die Menüpunkte Dashboard, Organisationen, Benutzer, Mail-Adressen, IP-Adressen und Audit-Log zur Verfügung. Welche Bereiche sichtbar und bearbeitbar sind, richtet sich nach der zugewiesenen Rolle.

Neue Organisationen, Benutzer sowie Mail- und IP-Einträge werden über die jeweiligen Listenansichten angelegt. Änderungen erfolgen über Bearbeitungsformulare, die Eingaben bereits beim Speichern validieren. Fehlerhafte Eingaben werden direkt am betroffenen Feld angezeigt. Dadurch kann die Pflege ohne technische Detailkenntnisse nachvollziehbar und sicher erfolgen.

Das Audit-Log dient der Rückverfolgung fachlicher Änderungen. Es zeigt, wann ein Datensatz erstellt, geändert oder gelöscht wurde. Soweit fachlich freigegeben, können Änderungen aus dem Audit-Log heraus zurückgesetzt werden. Ein kurzer Benutzerhinweis mit den wichtigsten Bedienregeln befindet sich zusätzlich im Anhang A.9: Kundenhinweis.

Für die Einführung beim Kunden ist keine umfangreiche Schulung erforderlich. Die Anwendung orientiert sich an den bekannten Objekten des Fachprozesses und verwendet eine durchgängige Bedienlogik für Listen-, Detail- und Bearbeitungsansichten. Dadurch kann die Kundendokumentation auf die für den Betrieb notwendigen Schritte konzentriert bleiben.

Im laufenden Betrieb ist darauf zu achten, dass Berechtigungen nur nach dem tatsächlichen Aufgabenbereich vergeben werden. Insbesondere administrative Rollen dürfen nicht pauschal verteilt werden, da sie organisationsübergreifende Pflegevorgänge ermöglichen. Neue Einträge für Mail-Adressen und IP-Adressen sollen vor dem Speichern fachlich geprüft werden, damit nur berechtigte Ausnahmen in den verwalteten Datenbestand aufgenommen werden.

6 Fazit

Mit dem *AntispamAdminCenter* wurde eine betriebsnahe Verwaltungsanwendung entwickelt, die den zuvor manuellen Pflegeprozess für Mail- und IP-Freigaben zentralisiert. Der wesentliche Nutzen liegt nicht nur in der Zeitersparnis, sondern vor allem in einer einheitlichen Datenhaltung, einer klaren Rechtevergabe und der Nachvollziehbarkeit fachlicher Änderungen.

Das Projekt hat gezeigt, dass auch ein fachlich begrenzter Anwendungsfall eine saubere technische Struktur erfordert, wenn mehrere Mandanten, Rollen und sicherheitsrelevante Vorgaben beteiligt sind. Für einen produktiven Ausbau sind insbesondere eine Anbindung an eine dedizierte Datenbank, weitergehende Deployment-Automatisierung und zusätzliche Integrationsschnittstellen für das Mail-Umfeld sinnvolle nächste Schritte.

Aus Sicht des Betriebs stellt das Ergebnis eine tragfähige Grundlage für die weitere Standardisierung des Prozesses dar. Die wesentlichen Risiken der Ausgangssituation, insbesondere fehlende Transparenz und uneinheitliche Pflegeabläufe, wurden durch die Projektdurchführung deutlich reduziert.

Literatur- und Quellenverzeichnis

Projektquellcode Eigener Projektordner PROJEKT mit den Bereichen `models`, `services`, `web` und `tests`.

Projektbeschreibung Eigene Projektnotizen sowie die technische Kurzdokumentation in der `README.md` des Projektordners.

FastAPI Offizielle Dokumentation von FastAPI für Routing, Request-Verarbeitung und Template-Integration.

SQLModel Offizielle Dokumentation von SQLModel für die Modellierung der Datenbankklassen.

Argon2 Herstellerdokumentation zur Passwort-Hashing-Bibliothek für die sichere Speicherung von Kennwörtern.

Persönliche Erklärung

Hiermit erkläre ich, dass ich die vorliegende Projektarbeit selbständig und nur unter Zuhilfenahme der aufgeführten Quellen angefertigt habe. Der vorgesehene zeitliche Rahmen wurde eingehalten.

Unterschrift des Prüfungsteilnehmers

A Anhang

A.1 Zeitnachweis

Tätigkeit	Stunden
Ist-Analyse und Anforderungskklärung	8
Soll-Konzept und Variantenvergleich	10
Entwurf von Datenmodell und Rollenmodell	8
Entwurf der Weboberfläche und Navigationsstruktur	4
Implementierung Stammdatenverwaltung	10
Implementierung Benutzer- und Rechteverwaltung	8
Implementierung Login-Schutz und Audit-Log	10
Tests, Fehlerkorrekturen und Abnahmevorbereitung	10
Kundendokumentation und Projektdokumentation	12
Summe	80

Tabelle 5: Zeitnachweis

A.2 Wirtschaftlichkeit

Position	Stunden	Satz	Kosten
Projektdurchführung	80	65€	5200€
Fachliche Begleitung und Abnahme	6	85€	510€
Softwarelizenzen	-	-	0€
Gesamt			5710€

Tabelle 6: Detailrechnung der Projektkosten

A.3 Variantenvergleich

Kriterium	Gewicht	Standardlösung	Eigenentwicklung
Passgenauigkeit	5	3	5
Mandantentrennung und Rechteabbildung	5	3	5
Nachvollziehbarkeit und Auditierung	4	3	5
Einführungsaufwand	3	3	4
Wartbarkeit im Betrieb	3	3	4
Gewichteter Nutzwert		51	80

Tabelle 7: Gewichteter Variantenvergleich

A.4 Use Case-Diagramm

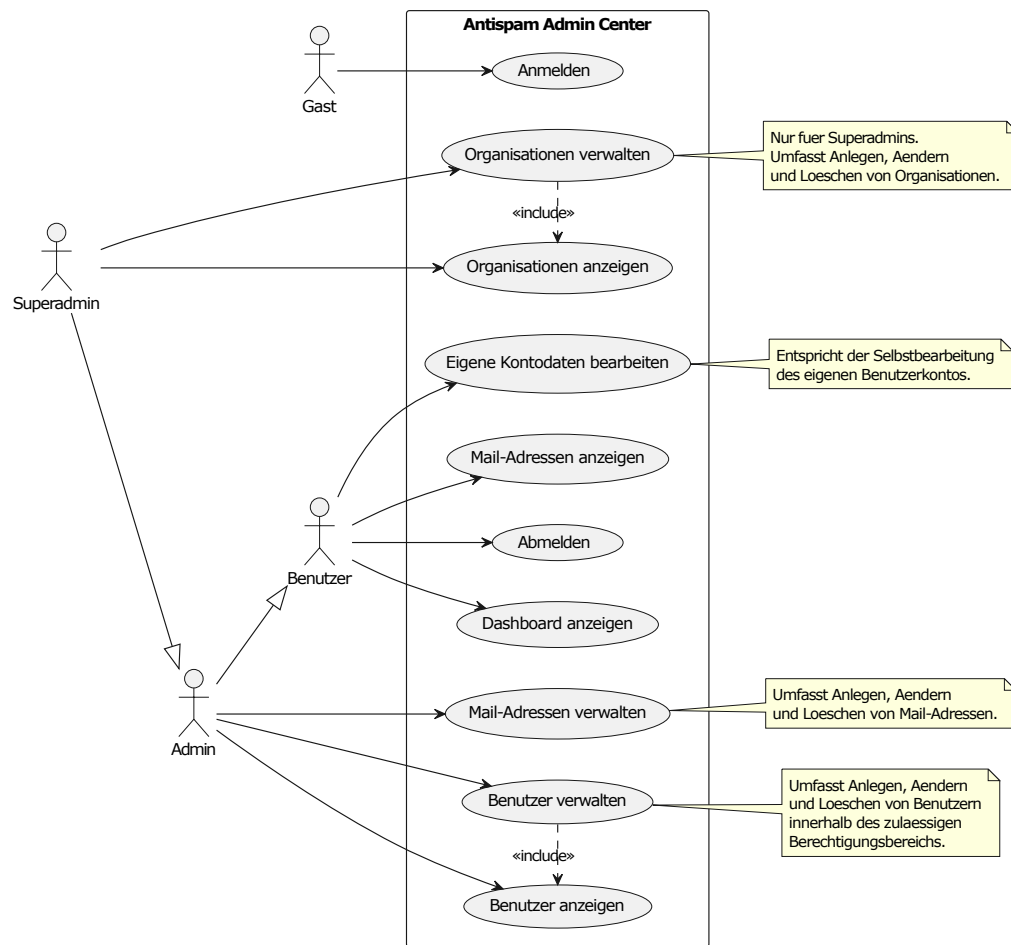


Abbildung 1: Use Case-Diagramm

A.5 Datenbankmodell

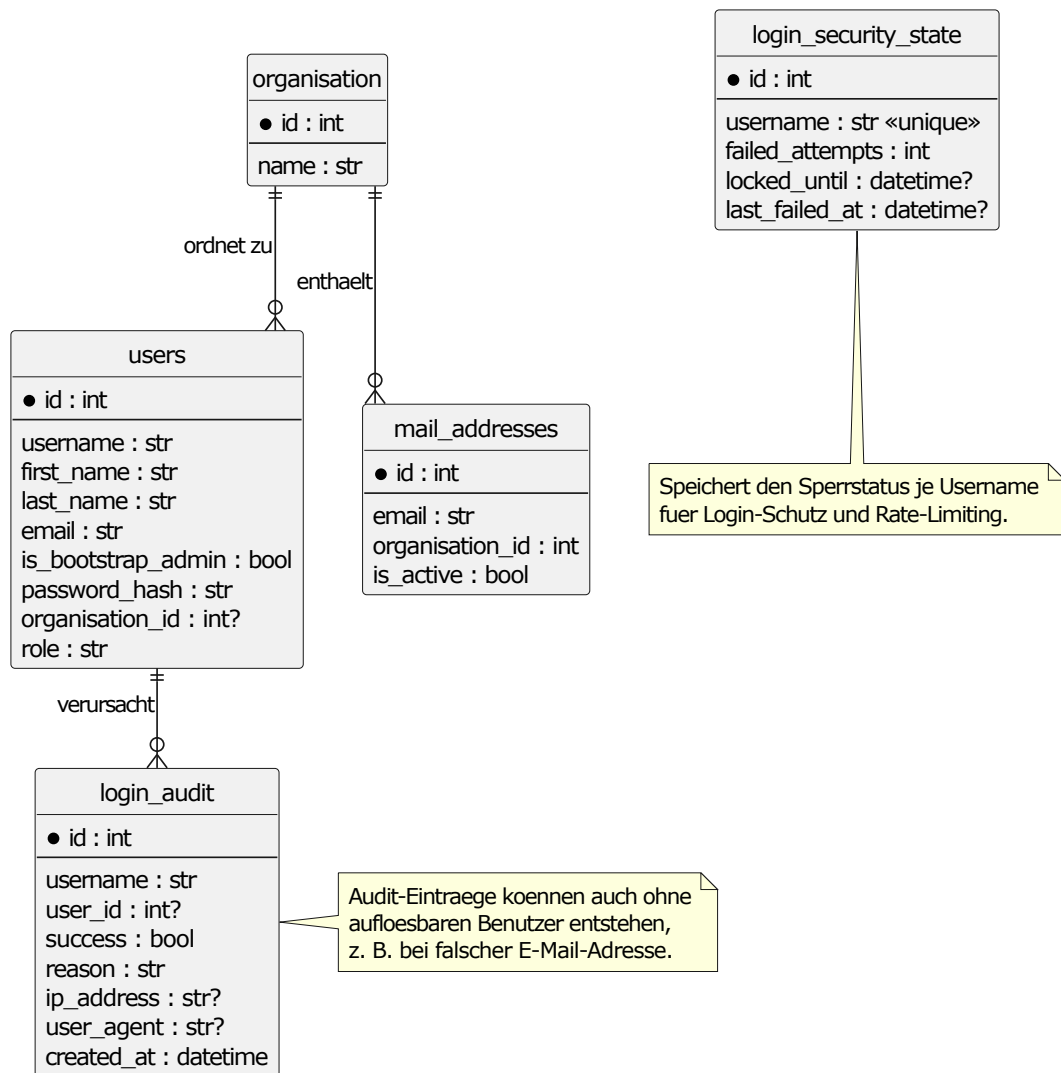


Abbildung 2: Datenbankmodell

A.6 Klassendiagramm

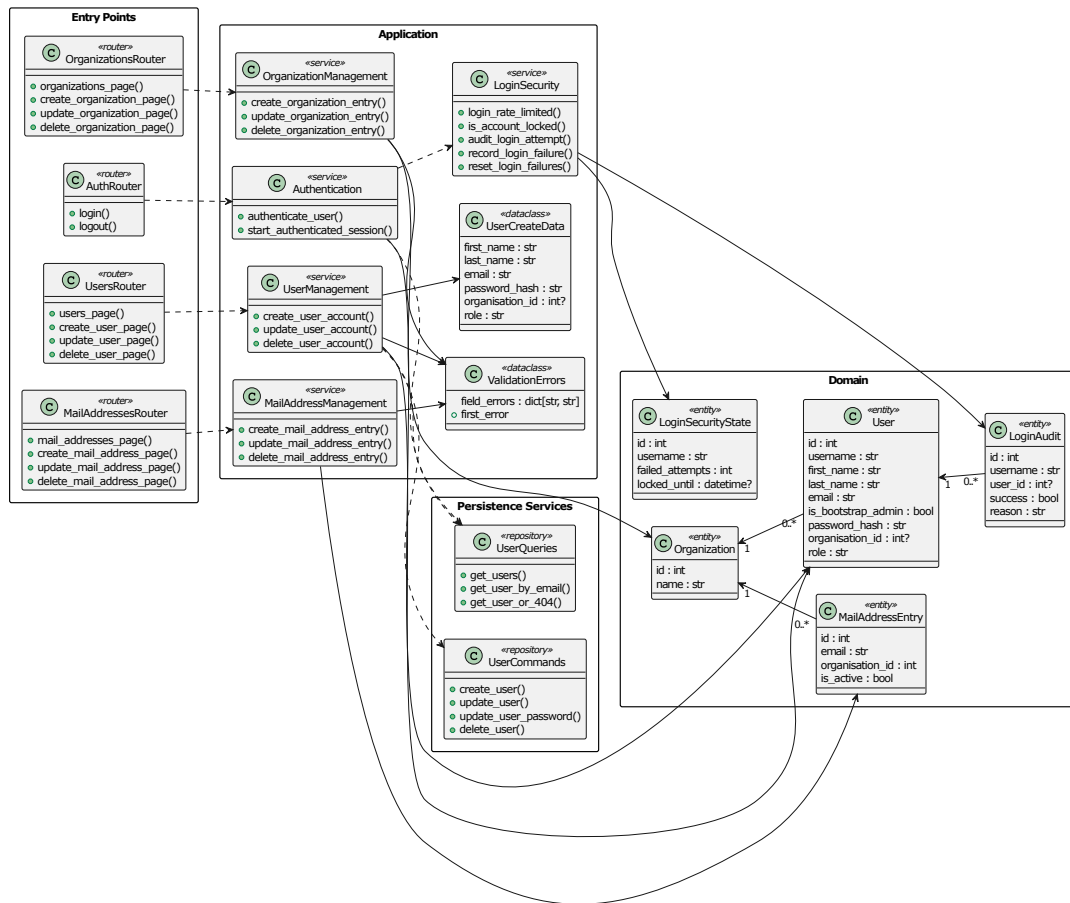


Abbildung 3: Klassendiagramm

A.7 Komponentendiagramm

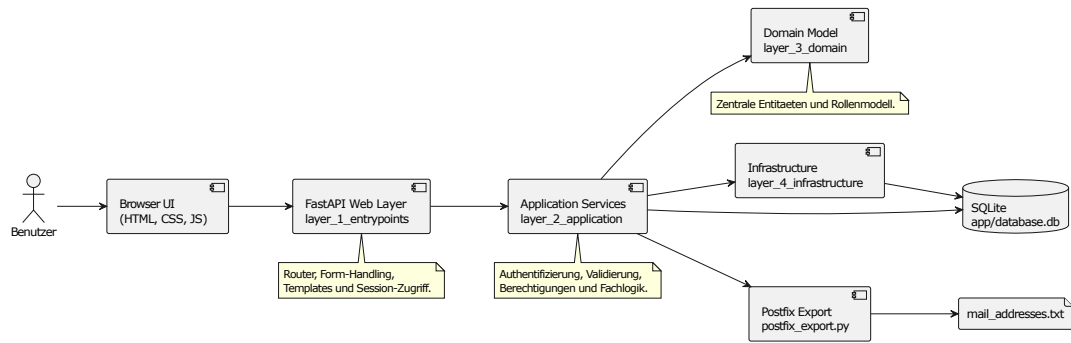


Abbildung 4: Komponentendiagramm

A.8 Sequenzdiagramm Login

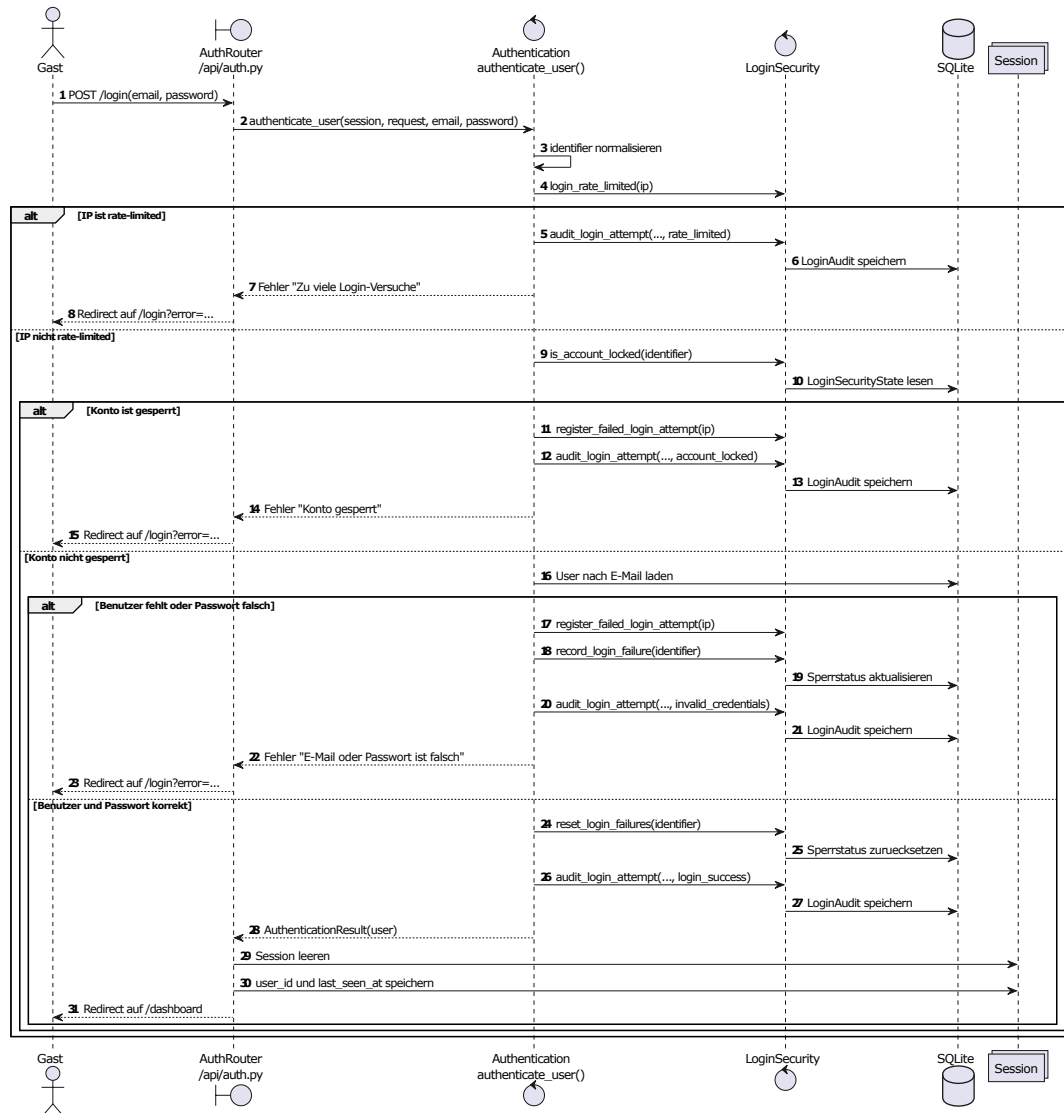


Abbildung 5: Sequenzdiagramm Login

A.9 Kundenhinweis

Die Anwendung ist für die tägliche Pflege von Organisationen, Benutzern, Mail-Adressen und IP-Adressen vorgesehen. Vor dem Löschen eines Datensatzes soll geprüft werden, ob der Eintrag noch für nachgelagerte Prozesse benötigt wird. Fachliche Änderungen sind über das Audit-Log nachvollziehbar.

A.10 Quelltextauszug Berechtigungslogik

```
1 from fastapi import HTTPException, status
2 from sqlmodel import Session, select
3
4 from app.core.config import ROLE_ADMIN, ROLE_ROOT, ROLE_USER
5 from app.models import IPAddressEntry, MailAddressEntry, Organization,
   User
6
7
8 def ensure_user_has_roles(current_user: User, *roles: str) -> User:
9     if current_user.role not in roles:
10         raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="
        Forbidden")
11     return current_user
12
13
14 def has_users(session: Session) -> bool:
15     return session.exec(select(User)).first() is not None
16
17
18 def is_root(user: User) -> bool:
19     return user.role == ROLE_ROOT
20
21
22 def is_admin(user: User) -> bool:
23     return user.role == ROLE_ADMIN
24
25
26 def is_manager(user: User) -> bool:
27     return user.role in {ROLE_ROOT, ROLE_ADMIN}
28
29
30 def get_accessible_organizations(
31     session: Session, current_user: User
32 ) -> list[Organization]:
33     if is_root(current_user):
34         return list(session.exec(select(Organization)).all())
35     if current_user.organization_id is None:
36         return []
37     organization = session.get(Organization, current_user.organization_id)
38     return [organization] if organization is not None else []
39
40
41 def get_accessible_mail_addresses(
42     session: Session, current_user: User
43 ) -> list[MailAddressEntry]:
44     statement = select(MailAddressEntry)
```

```

45     if not is_root(current_user):
46         statement = statement.where(
47             MailAddressEntry.organization_id == current_user.organization_id
48         )
49     return list(session.exec(statement).all())
50
51
52 def get_accessible_ip_addresses(
53     session: Session, current_user: User
54 ) -> list[IPAddressEntry]:
55     statement = select(IPAddressEntry)
56     if not is_root(current_user):
57         statement = statement.where(
58             IPAddressEntry.organization_id == current_user.organization_id
59         )
60     return list(session.exec(statement).all())
61
62
63 def get_accessible_users(session: Session, current_user: User) -> list[
64     User]:
65     if current_user.role == ROLE_USER:
66         return [current_user]
67     statement = select(User)
68     if not is_root(current_user):
69         statement = statement.where(User.organization_id == current_user.
70             organization_id)
71     return list(session.exec(statement).all())
72
73
74 def get_manageable_organizations(
75     session: Session, current_user: User
76 ) -> list[Organization]:
77     return get_accessible_organizations(session, current_user)
78
79
80 def ensure_manageable_organization(
81     current_user: User, organization_id: int | None
82 ) -> None:
83     if is_root(current_user):
84         return
85     if current_user.organization_id is None or current_user.organization_id
86         != organization_id:
87         raise HTTPException(
88             status_code=status.HTTP_403_FORBIDDEN,
89             detail="Forbidden",

```

```

90 def ensure_manageable_mail_address(current_user: User, entry:
    MailAddressEntry) -> None:
91     ensure_manageable_organization(current_user, entry.organization_id)
92
93
94 def ensure_manageable_ip_address(current_user: User, entry: IPAddressEntry
    ) -> None:
95     ensure_manageable_organization(current_user, entry.organization_id)
96
97
98 def ensure_manageable_user(current_user: User, target_user: User) -> None:
99     if target_user.is_root:
100         raise HTTPException(
101             status_code=status.HTTP_403_FORBIDDEN,
102             detail="Forbidden",
103         )
104     if is_root(current_user):
105         return
106     if target_user.role == ROLE_ROOT:
107         raise HTTPException(
108             status_code=status.HTTP_403_FORBIDDEN,
109             detail="Forbidden",
110         )
111     ensure_manageable_organization(current_user, target_user.
        organization_id)

```

Listing 1: Berechtigungslogik für organisationsbezogene Zugriffe

A.11 Testauszug

```
1 import pytest
2 from fastapi import FastAPI
3 from fastapi.testclient import TestClient
4 from sqlmodel import SQLModel, Session, create_engine, select
5
6 from app.core.config import ROLE_ADMIN
7 from app.models import Organization, User
8 from app.web.dependencies import get_current_user_dependency, get_session
9 from app.services.passwords import verify_password
10 from app.web.api.router import api_router
11
12
13 @pytest.fixture
14 def session(tmp_path):
15     database_path = tmp_path / "test_users_api.db"
16     engine = create_engine(f"sqlite:/// {database_path}")
17     SQLModel.metadata.create_all(engine)
18
19     with Session(engine) as db_session:
20         yield db_session
21
22
23 @pytest.fixture
24 def client(session: Session):
25     app = FastAPI()
26     app.include_router(api_router)
27
28     def override_get_session():
29         yield session
30
31     current_user_holder = {"user": None}
32
33     def override_get_current_user():
34         return current_user_holder["user"]
35
36     app.dependency_overrides[get_session] = override_get_session
37     app.dependency_overrides[get_current_user_dependency] =
38         override_get_current_user
39
40     with TestClient(app) as test_client:
41         yield test_client, current_user_holder
42
43 def test_create_user_endpoint_redirects_and_persists(client, session:
44     Session):
45     test_client, current_user_holder = client
```

```

45 organization = Organization(name="Test_Organisation")
46 session.add(organization)
47 session.commit()
48 session.refresh(organization)
49
50 current_user_holder["user"] = User(
51     id=999,
52     username="admin@example.de",
53     first_name="Admin",
54     last_name="User",
55     email="admin@example.de",
56     password_hash="admin-passwort",
57     organization_id=organization.id,
58     role=ROLE_ADMIN,
59 )
60
61 response = test_client.post(
62     "/users/create",
63     data={
64         "first_name": "Max",
65         "last_name": "Mustermann",
66         "email": "max@example.de",
67         "password": "StarkesPasswort1!",
68         "organization_id": str(organization.id),
69         "role": ROLE_ADMIN,
70     },
71     follow_redirects=False,
72 )
73
74 created_user = session.exec(
75     select(User).where(User.email == "max@example.de")
76 ).one()
77
78 assert response.status_code == 303
79 assert response.headers["location"] == "/users/page"
80 assert created_user.first_name == "Max"
81 assert created_user.last_name == "Mustermann"
82 assert created_user.username == "max@example.de"
83 assert verify_password("StarkesPasswort1!", created_user.password_hash)
84
85
86 def test_edit_user_endpoint_redirects_and_updates(client, session: Session
87 ):
88     test_client, current_user_holder = client
89     organization = Organization(name="Test_Organisation")
90     session.add(organization)
91     session.commit()
92     session.refresh(organization)

```

```

92
93     existing_user = User(
94         username="max@example.de",
95         first_name="Max",
96         last_name="Mustermann",
97         email="max@example.de",
98         password_hash="altes-passwort",
99         organization_id=organization.id,
100        role=ROLE_ADMIN,
101    )
102    session.add(existing_user)
103    session.commit()
104    session.refresh(existing_user)
105
106    current_user_holder["user"] = User(
107        id=999,
108        username="admin@example.de",
109        first_name="Admin",
110        last_name="User",
111        email="admin@example.de",
112        password_hash="admin-passwort",
113        organization_id=organization.id,
114        role=ROLE_ADMIN,
115    )
116
117    response = test_client.post(
118        f"/users/{existing_user.id}/edit",
119        data={
120            "first_name": "Moritz",
121            "last_name": "Musterfrau",
122            "email": "moritz@example.de",
123            "password": "NeuesPasswort1!",
124            "organization_id": str(organization.id),
125            "role": ROLE_ADMIN,
126        },
127        follow_redirects=False,
128    )
129
130    updated_user = session.get(User, existing_user.id)
131
132    assert response.status_code == 303
133    assert response.headers["location"] == "/users/page"
134    assert updated_user is not None
135    assert updated_user.first_name == "Moritz"
136    assert updated_user.last_name == "Musterfrau"
137    assert updated_user.email == "moritz@example.de"
138    assert updated_user.username == "moritz@example.de"
139    assert verify_password("NeuesPasswort1!", updated_user.password_hash)

```

```

140
141
142 def test_delete_user_endpoint_deletes_user(client, session: Session):
143     test_client, current_user_holder = client
144     organization = Organization(name="Test_Organisation")
145     session.add(organization)
146     session.commit()
147     session.refresh(organization)
148
149     existing_user = User(
150         username="max@example.de",
151         first_name="Max",
152         last_name="Mustermann",
153         email="max@example.de",
154         password_hash="passwort",
155         organization_id=organization.id,
156         role=ROLE_ADMIN,
157     )
158     session.add(existing_user)
159     session.commit()
160     session.refresh(existing_user)
161
162     current_user_holder["user"] = User(
163         id=999,
164         username="admin@example.de",
165         first_name="Admin",
166         last_name="User",
167         email="admin@example.de",
168         password_hash="admin-passwort",
169         organization_id=organization.id,
170         role=ROLE_ADMIN,
171     )
172
173     response = test_client.post(
174         f"/users/{existing_user.id}/delete",
175         follow_redirects=False,
176     )
177
178     deleted_user = session.get(User, existing_user.id)
179
180     assert response.status_code == 303
181     assert response.headers["location"] == "/users/page"
182     assert deleted_user is None

```

Listing 2: Beispielhafte API-Tests für die Benutzerverwaltung